

AI in Power Plant: From Data to Decisions

Participant course material — internal faculty programme, Koradi TPS
(3×660 MW) · S. H. Parihar

Table of Contents

How to use this document

This is the participant material for the one-day internal faculty programme **AI in Power Plant: From Data to Decisions**, delivered at Koradi Thermal Power Station. It is not a transcript of the slides. The slides carry the minimum needed in the room; this document carries the working — the arithmetic, the worksheets for each lab, the honest caveats, and the reference tables you will want a week later when you try any of this on your own unit.

Everything hands-on lives at one address:

pariharshyamu.github.io/simulator

Open it on your phone. Every artefact — the Algorithm Anatomy Theatre, the Model Builder, the Predictive Maintenance Simulator, the AI Simulation Lab and Pocket RAG — runs entirely in the browser, works on a phone screen, needs no login, and sends nothing anywhere. Once a page has loaded it keeps working offline. The labs in this document assume you have it open.

Two rules govern every number in this material. First, every rupee and every plant figure comes from documents this station already signs: the June 2026 energy bill, the provisional Part-I FSA bill with the F10 ECR calculation, and the MERC merit-order dispatch stack for July 2026. Nothing is estimated, benchmarked from a brochure, or vendor-supplied. Second, the machine data in the labs is **synthetic** — generated from a physical model with the fault injected on a known day — and is labelled as such wherever it appears. That is not a weakness. Real historian data never tells you the true onset day of a fault, so a detection claim can never be marked against truth. Synthetic data is the only data with an answer key, and a teaching course that wants to be honest about what models can and cannot do needs the answer key.

The day is built on one spine. A single reading — **74.2 °C on the drive-end bearing of an ID fan** — enters at 10:35 and is followed through every algorithm, every lab and every block until it becomes a signed work order. By 17:00 you will have run every step of that journey yourself, on your own phone, and voted on the decision at the end of it.

Chapter 1 — From plant to data

1.1 The asset you already own

A single large rotating machine — a fan, a pump, a mill — carries something like forty useful tags: bearing temperatures, winding temperatures, vibration, currents, flows, differential pressures, and the duty and ambient context around them. At a one-minute scan rate that is roughly **21 million readings per machine per year**, already collected, already time-stamped, already paid for, sitting in the historian.

This is the single most important fact in the course. AI in a power plant is about 80% this asset and 20% algorithms. The algorithms are free and interchangeable; the asset is unique to us, and its quality — scan rates, compression settings, dead sensors, calibration discipline — is decided by engineers, not by data scientists. The first AI project at this station is an export, not a purchase order.

1.2 One reading means nothing

Is 74.2 °C on a drive-end bearing fine or frightening? The only correct answer is: *it depends*. At 45% load on a January night it is alarming. At full load on a May afternoon, with cooling water at 31 °C, it is ordinary. The reading carries no meaning without its context — load, ambient, cooling-water temperature — and no fixed threshold can supply that context. A 90 °C alarm is simultaneously too late (a machine can be badly degraded long before 90 °C) and too early (a healthy machine can brush 88 °C on the worst afternoon of May, and an alarm that cries wolf in summer is disabled by October).

1.3 The one idea of the day

Everything in this course that is called “AI” is one subtraction, dressed four different ways:

1. **Measure** — what the sensor reads now: 74.2 °C.
2. **Expect** — what history says the reading *should* be for this load, this ambient, this cooling-water temperature: 71.9 °C.
3. **Subtract** — residual = actual – expected = **+2.3 °C**. The duty and the weather have been removed. What is left is the machine.
4. **Decide** — a residual that stays outside its band for days is evidence. Evidence goes to a person, and the person decides.

The sentence to keep: **a flat residual means a healthy machine, whatever the weather is doing**. Every algorithm in Chapter 2 — nearest neighbours, principal components, neural networks — is nothing more than a different way of computing step 2.

1.4 What the raw tag hides

Plot the bearing tag’s monthly mean across a year and it rides the seasons: up through the pre-monsoon heat, down when the monsoon arrives, up again in October. Now inject a real fault in late July. For roughly two months, the faulted machine’s readings stay *inside* the envelope a healthy twin sweeps through its own normal year. The fault is real, growing, and completely

invisible to the eye and to any threshold — because the seasonal swing is larger than the early fault. Making that hidden gap visible, early, is the entire subject of this course.

Chapter 2 — Neural networks, opened up

Lab vehicle: the Algorithm Anatomy Theatre. Modules 3 and 4 are run live in the session; Modules 1 (k-NN), 2 (PCA) and 6 (k-means) are homework with the same one reading flowing through them. Module 5 (retrieval) returns in Chapter 7.

2.1 Three ways machines learn — the whole vocabulary

Supervised learning maps inputs to outputs from examples with answers: given load, ambient and CW temperature, predict what this bearing should read. **Unsupervised** learning finds structure without answers: cluster a year of operation into regimes — start-up, low load, monsoon, summer full load — nobody labelled. **Reinforcement** learning learns actions by trial and reward; it is powerful, and it belongs nowhere near a live unit today. Almost everything in this course is supervised learning of an expected value. Keep the vocabulary that small and no vendor deck will disorient you.

2.2 The simplest model that works

Before any network, fix the baseline. To know what the bearing should read now, find the **12 most similar past hours** — similar load, similar ambient, similar cooling water — and average what the bearing read then. That average is the expected value; subtract, and you have a residual. No training, no weights, and fully explainable to a shift engineer: “you ran like this on these twelve days, and you were cooler then.” This is k-nearest neighbours, it is Module 1 of the Theatre, and most commercial condition-monitoring products are exactly this with better packaging. Any fancier model must beat it to justify its complexity.

2.3 Gradient descent — how every model is fitted

Lab worksheet — Theatre Module 3 (12 minutes). Open the Theatre, pick Module 3, and run the acts. The error surface is drawn as terrain: every position is a candidate model, height is how wrong it is, and training is a ball rolling downhill. Then break it: drag the learning rate up until the ball overshoots the valley and climbs the far wall — divergence, seen rather than described. Find the largest step size that still converges; that tension (large steps train fast until they explode) is real and is tuned by hand in every serious project.

What training *means*, in one line:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \cdot \partial L / \partial \mathbf{w}$$

Each dial (weight) w moves a small step η against its own share of the blame for the current error. Repeat over every hour of history until the error stops falling. That is the whole of “the model learns”.

2.4 A neural network, from the inside

Lab worksheet — Theatre Module 4 (15 minutes). The network on screen is real: 8 inputs → 10 hidden → 6 hidden → 1 output, which is **146 weights and 17 biases = 163 parameters**, trained live in your browser on the healthy hours of the synthetic year. Run the six acts: the wiring, eight numbers arriving, the forward pass, the error, the backward pass, training. Then **tap any sphere**. The panel opens that neuron's complete arithmetic at the current hour — its weights, its weighted sum, its activation.

One neuron does this and nothing else:

$$z = \sum_i w_i x_i + b \text{ then } a = \tanh(z)$$

Multiply each input by its own weight, add them up, add a bias, squash the total into $(-1, 1)$. A “layer” is a row of these; the network is three rows. There is no further mystery to find.

The backward pass is the chain rule sharing out blame. At the output, $\delta = 2(\hat{y} - y)$. Each earlier neuron receives its share of δ through the weights that connect it forward, multiplied by the derivative of its own activation — for $\tanh$, $1 - a^2$. Then every weight updates by the line in §2.3. Note what the derivative implies: a neuron whose output is pinned near ± 1 has derivative near zero — *it has stopped learning*, however wrong the answer is. That is the vanishing gradient, and you can find a saturated neuron yourself in the module.

Why the bend matters. Remove every activation and a stack of linear layers — any depth — multiplies out to a single straight line. The squash at each neuron is what lets a network represent a curve. Chapter 3 contains a real curve (the cooler knee) that no straight-line model can learn, which is the practical reason this matters.

2.5 The first sanity check on any model

Module 4 computes, live, the sensitivity of the output to each input: how many degrees the prediction moves per unit of each tag. Load and ambient dominate — exactly as the physics says they should. That agreement is the check. **On any supplied model, ask for the sensitivity table first.** If the largest sensitivity belongs to a tag with no physical route to the target, the model has learned a coincidence of its training window, and it will betray you the first time the coincidence breaks.

2.6 What a network is, and is not

It **is**: a flexible curve-fitter; inspectable (every weight is a printable number); retrainable in seconds at this size. It **is not**: a model of the machine (it contains no physics); reliable outside the range of data it saw; “explainable” in the sense a manager means the word. For one bearing on one fan, k-NN or PCA is usually the better tool — simpler, hungrier for less data, failing more visibly. Networks earn their keep where relationships are strongly non-linear and history is long: combustion, mill throughput, load-following steam temperatures. **Choose the smallest model that works.** That is an engineering decision, not a fashion decision.

Chapter 3 — Data synthesis, and building your own model

Lab vehicle: the Model Builder — drag-and-drop (tap-to-add on a phone) blocks that assemble and train a real neural network in about 1.5 seconds, on the same synthetic year the simulator uses. Tap any block and it opens into the exact arithmetic it performs, with the live model's numbers substituted.

3.1 Why synthetic data, and how the year is built

A teaching fault needs a birthday. Real data never confesses one, so every lab runs on a synthetic year — 8,760 hourly rows — built from a physical model whose honesty is worth two minutes:

Signal	How it moves	Why it matters
Unit load	Mean-reverts around 80%; nights -13%, Sundays -10%, season $\pm 6\%$	Duty is the biggest driver of every temperature on the machine
Ambient	Monthly means: Jan 21 °C, rising to May 33.5 °C , then a monsoon drop to 25.5 °C, lifting in October	Not a sine wave. One sinusoid cannot make an Indian year — and neither can a model that never saw one
Cooling water	Follows ambient with ~9 days of lag	A basin has thermal inertia; the knee below hangs off this signal
Fault severity	$((t - \text{onset})/\text{tff})^{\text{shape}}$ — for the fan: onset day 200, 110 days to failure, shape 1.7	Power-law growth: convex, quiet early, steep late

Two consequences deserve their own paragraphs.

The convexity of failure. Thirty days into the fault, severity is 11%. At half its life, barely a third. The first third of a fault produces almost no signal, which is why nobody catches these by eyeballing trends, and why every “why did detection take so long” question in Chapter 4 has the same answer: the early fault was genuinely quiet.

The knee. The lube-oil cooler is sized for a design cooling-water temperature of 27.5 °C. Below it, oil outlet temperature tracks ambient almost linearly. Above it, the cooler runs out of approach and the generator adds $1.35 \times (\text{CW} - 27.5)^{1.45}$. Fit a straight line to January and February — cooling water at 19–22 °C, entirely below the knee — and it extrapolates confidently into May and is badly wrong, because it never saw the knee exist. This single term is why the dataset must be a year long, and why Chapter 2's “bend” is not academic.

3.2 What synthesis cannot teach

Real exports are ugly, and the Builder's CSV reader is built for the ugliness: semicolon separators, decimal commas (“61,4”), status strings (“I/O Timeout”, “Bad”, “Shutdown”) sitting inside numeric columns, flatlined sensors, historian compression that smooths real variation

away. Each pathology poisons a model in its own way — a flatline teaches the model the machine is immortal; coerced status strings teach it the machine periodically freezes solid. Drop one of our own unit's exports into the Builder tomorrow: it will parse it, profile every column, and tell you which tags are usable before any training happens.

3.3 Lab worksheet — the four experiments (20 minutes)

The default model predicts the DE bearing temperature from **duty and weather only** — fan motor current, flue gas flow, ambient. That choice is deliberate: it is the honest configuration, and the experiments below each break it in an instructive way. Record your numbers as you go.

Experiment 1 — the honest default. Press Train. Read the verdict panel: $\sigma \approx 2.05$ °C (of which sensor noise is only 0.89), band ± 8.2 °C at 4σ held 72 hours, fault begins day 200, advisory **day 255** — 55 days to be sure, one episode all year, and on the day it fired the fault was only **31% of the way to failure**. Then tap the blocks open and read the arithmetic: the same equations as Chapter 2, with your model's numbers in them.

Experiment 2 — the winter-only window. Set the training window to days 0–45 and retrain. The winter model fits its own window beautifully ($\sigma \approx 0.61$ °C — tighter than the honest model!) and then runs **+2.4 °C wrong on average through the pre-monsoon, four times its own alert band, nineteen times worse** than the season-spanning model. It never met the knee. Every bad model looks good on its own training data.

Experiment 3 — the cardinal sin. Set the window to days 0–300, so it contains the first hundred days of the fault. The advisory slides from day 255 to **day 315** — sixty days of warning gone — because the model absorbed the early degradation as normal and widened its own band to match. Set the window to 0–330 and the advisory **never comes at all**. Nothing on the loss curve warns you. This is the single most common way real condition-monitoring programmes die.

Experiment 4 — buy back 25 days with one tag. Tick the lube-oil cooler outlet temperature into the inputs and retrain. σ collapses from 2.05 to 0.60 °C — the knee is now explained — the band tightens from ± 8.2 to ± 2.4 °C, and the advisory moves from day 255 to **day 230**. Twenty-five days of warning, bought by knowing where the failure starts. **Input choice beats architecture.** The engineer who knows the plant beats the data scientist who knows the library.

3.4 Where the alert band comes from

σ is the model's root-mean-square error on its own training window — the model's *ignorance*, not the instrument's noise. The Builder splits it on every run: of the default model's 2.05 °C, only 0.89 °C is irreducible hour-to-hour sensor noise; the remaining ~1.8 °C is structure the inputs cannot explain (mostly the knee). **The alarm band is a property of the model, not the machine.** Improve the model and the band narrows, and days of warning appear from nowhere — Experiment 4 is the proof.

One trap deserves a warning box of its own. Predict a tag the fault never touches — fan motor current — from tags the fault *does* move (bearing temperature, vibration), and after onset the residual runs **22 A negative** on a machine whose current never changed. A negative residual

does not mean the reading fell; it means the model's *expectation* rose. In the healthy window, the only thing that lifted those inputs was load, so the model learned "hot and shaking = working hard = drawing more current". A fault-contaminated *input* is the mirror image of target leakage, and it points the investigation at the wrong machine.

3.5 Three rules for a training window

1. **Span the seasons.** A window that has not met May does not know May exists. One full year, minimum, in this climate.
2. **End before the period you judge.** Procedurally, always, no exceptions. Nothing in the training metrics will catch this for you.
3. **Duty in, symptoms out.** Inputs are what drives the machine — load, ambient, CW. A symptom tag as an input teaches the model to expect the fault.

And the standing rule: **retrain after every overhaul**. Maintenance legitimately changes what "normal" is; a model that remembers the old bearing will alarm on the new one.

Chapter 4 — Predictive maintenance, end to end

Lab vehicle: the PdM Simulator — eight stages from asset selection to signed work order, across five machines, with a 3-D model that degrades as you scrub through the year.

4.1 From one model to a programme

Chapter 3 built stage 6 of an eight-stage pipeline. A predictive maintenance *programme* is all eight:

01 Asset & failure mode · 02 Instrumentation · 03 Data acquisition · 04 Data quality · 05 Feature engineering · 06 Model training · 07 Validation · 08 Inference & value

Most failed programmes die in stages 1–4, before any AI exists to blame: wrong failure mode, missing sensor, dead data, unexamined quality. No model recovers from these. The glamour lives on the right of the pipeline; the leverage lives on the left, and the left is engineering.

The simulator carries five cases — ID fan bearing, HT motor rotor bar, coal mill wear, BFP seal and cavitation, transformer cooling — each a synthetic year with a known fault day and a deliberate trap (an uninstrumented sensor, a tag that would leak the answer), because real programmes meet exactly these. The fan is the guided case; the other four are homework with answer keys.

4.2 Stages 1 and 2 — name the failure mode, audit the instruments

For the fan the chain is specific: the cooling-water side of the lube-oil cooler fouls → oil runs hot → the film in the journal bearing thins → babbitt wears → metal temperature climbs to the 100 °C trip. A model with no named failure mode is a dashboard: colourful, comprehensive, incapable of answering "what would we have caught, and when?"

The instrument audit follows the chain. Eight tags exist on the fan; the ninth — cooling-water flow through the cooler, the tag *closest to the mechanism* — is not fitted, which is entirely typical. Remember Experiment 4: the mechanism-adjacent tag was worth 25 days of warning. **A ₹50,000 sensor can outperform a ₹50 lakh analytics platform.** Audit before you model, and ask which tags your own worst actor is missing.

4.3 The P-F curve, with measured days

Every rung below was measured in the simulator on the known-onset case — not copied from a textbook:

Detection rung	Day	Warning before failure
Fault begins (P)	200	—
Residual with mechanism tag (Exp. 4)	~230	~80 days
Residual, duty & weather (default)	255	~55 days
Vibration trend confirms	~280	~30 days
Fixed 90 °C alarm	~290	~20 days
Trip / failure (F)	~310	0

The gaps between rungs are the value, in days, of each improvement — a sensor, a model, a discipline. None is free, and each buys a measurable number of days on the same fault.

Lab worksheet — scrub the year (12 minutes). Open the fan case and drag the day slider slowly from 0 to 365, watching the 3-D machine: the bearing warms through its palette, the cut-away shows the film thinning, the sensor dots carry live values. Decide, by eye alone, on which day *you* would have called maintenance, and write it down. Across a room the answers spread from roughly day 230 to day 330 — and that spread is the lesson. Eyeballing a trend is a lottery; the residual's day 255, with one episode a year, is what consistency looks like.

4.4 Stages 4 and 5 — quality gates and features

Every tag gets a quality profile before it is allowed near training: freeze detection, gap census, range checks, timestamp audit. The gates catch the five quiet liars — flatlines, zero-filled gaps, historian compression, calibration drift, clock skew between systems.

Features are where plant knowledge enters. Feed the model: load- and ambient-normalised temperatures; the **DE – NDE difference** (it cancels common-mode drift *and* separates sensor fault from machine fault — a sensor fault moves one bearing, a machine fault moves both — the thirty-second triage every operator can do); the oil ΔT across the cooler; seven-day rolling means and slopes. Refuse the traps: hour-of-day (correlates with everything, means nothing), day-number (encodes the answer outright), any symptom tag as an input (§3.4), the twin bearing as an input (the model copies it and learns nothing).

4.5 Stage 7 — validation as an alert budget

Validation is not an accuracy percentage; it is an **alert budget**. At 4σ held 72 hours, the fan case yields one episode a year and 55 days of warning. Loosen to $2\sigma/24$ h and warning improves marginally while episodes explode into dozens — and a control room ignores the forty-first false page. Decide the budget first — *how many investigations a month will maintenance actually staff?* (two per month per unit is a defensible start) — then spend it with σ and persistence.

Two checks make a validation honest. The **sister machine**: run the same model on the healthy twin, judged by its own threshold — if it alarms on the twin, it learned the weather, not the fault. (The threshold must come from the sister’s own quiet period; applying machine A’s band to machine B fakes the nuisance rate to zero, a trick worth knowing when reading vendor claims.) And **remaining useful life as a band, not a date**: “fails 14 March” is astrology that will be remembered against the programme; “90% confident it survives past the outage window, and here is what moves the odds” is engineering that planning can act on — advance a window, stage spares, derate on hot afternoons.

4.6 Stage 8 — the advisory, and the vote

An advisory is a claim with evidence, addressed to an engineer — not an alarm addressed to nobody. The form matters: the claim, the residual behind it, the mechanism hypothesis, an independent second witness, the RUL band, and a recommended action with a window. Everything inspectable; nothing that says “the AI decided”.

The session ends the block with a vote on day 255: sign the work order ($\approx$ ₹35 lakh, 0.4-day planned window, cooler inspection) or wait for more evidence (severity $\sim 45\%$ and convex, consequence odds roughly 10:1 against a ≈ 2.5 -day forced outage costing ₹4+ crore with replacement energy). The room splits, and the split is the point: **this moment is the decision layer**, and everything in the course — tighter bands, earlier days, second witnesses — exists to make this moment easier. At our June availability of 72.5%, every avoided forced outage is availability we visibly need. PdM’s money is in availability, not in the maintenance budget it is usually asked to justify against.

(The outage arithmetic above is the simulator’s scenario, stated as such — the ratio, not the specific rupees, is the durable lesson.)

Chapter 5 — Our own numbers: efficiency and optimisation

Lab vehicle: the AI Simulation Lab — SIM-2 (heat rate attribution), SIM-3 (auxiliary power optimiser) and SIM-6 (combustion multi-objective optimiser) run live; SIM-4 (GCV soft sensor) and SIM-5 (RUL) are homework.

5.1 Koradi 8–10, June 2026, exactly as filed

Metric	Actual	Norm	Gap
Availability / PLF	72.5% / 66.62%	—	964.8 MU gross
Net heat rate	2,442.18 kcal/kWh	2,230	+212.18 kcal/kWh
Auxiliary power	6.95%	6.0%	+0.95 pp $\approx$ 9.17 MU
As-fired GCV	3,061 kcal/kg	—	—
Variable cost	₹3.378/kWh	MOD ₹3.284	+9.4 paise
Secondary fuel oil	0.41 ml/kWh	0.50	better than norm
Transit loss	0.49%	0.80%	better than norm

The net heat-rate gap alone was worth **₹25.61 crore in June**.

5.2 The decomposition that changes the conversation

Net heat rate already **contains** auxiliary power, so decompose before diagnosing, using the identity **gross HR = net HR $\times$ (1 – aux)**:

- **Boiler-turbine (gross) gap: 176.25 kcal/kWh = ₹22.87 crore** — gross actual 2,272.45 against a 2,096.2 norm. A genuine thermodynamic shortfall.
- **Auxiliary excess: 0.95 points = ₹3.01 crore** — a separate, smaller, electrical-side problem.

Koradi 8–10 is the mirror image of Nashik, whose boiler and turbine *beat* the norm while the entire penalty sits in auxiliary consumption. Different disease, different medicine — and any AI effort here must be pointed at the thermodynamic gap, because that is where our money is.

Never add the two numbers. The ₹25.61 crore net-side figure and the ₹3.01 crore aux-side figure overlap by construction; adding them counts the auxiliary consumption twice. Anyone who quotes “nearly ₹29 crore of addressable loss” has invented several crore a month out of double counting — in front of a regulator, eventually.

5.3 Lab worksheet — SIM-2, finding homes for 176 kcal (12 minutes)

Open SIM-2 on the Koradi 8–10 row. The sliders are the classical loss accounts — condenser back pressure, HP heater out of service, spray flows, auxiliary — and the task is to find combinations that plausibly explain our 176 kcal/kWh gross gap, watching the rupee counter convert every kcal to crore per month. Record your combination, then compare with a neighbour’s. You will not agree — **several different combinations explain the same bill** — and that is the finding. The monthly bill cannot arbitrate between explanations; only continuous, tag-level loss accounting can, and that is precisely the honest AI project our numbers are asking for.

Where a gap like ours usually lives, honestly split: condenser vacuum and CW performance (the classic largest account — largely controllable); cycle isolation and HP heaters (find it, fix it, verify the kcal returned); SH/RH sprays (partly controllable); and part-load operation at 66% PLF on merit order (real, significant, and **structural** — no operational cleverness recovers it, and an honest programme says so).

5.4 Lab worksheet — SIM-3, auxiliary power as a scheduling problem (10 minutes)

The optimiser's arithmetic, from the simulation's equipment models: a mill draws 340 kW plus 2.6 kW per t/h above 30; each CW pump is 1,650 kW; every energised ESP field is 26 kW; ID fan power goes roughly as the cube of flow. Auxiliary excess is therefore mostly a **counting problem** — how many mills, pumps and fields for this load, this coal, this season — under safety and process constraints.

Scoreboard rules: take the default scenario, choose mill count, CW pumps and ESP fields, and beat the baseline house load without tripping a feasibility flag. The recurring discovery — one CW pump more than needed at part load in cool months — is exactly the kind of item hiding inside our 0.95 excess points, and it needs no capital to collect.

5.5 Lab worksheet — SIM-6, combustion on the Pareto frontier (8 minutes)

There is no single best combustion setting; there is a frontier of defensible ones. In SIM-6's model, NO_x rises roughly 78 mg/Nm³ per point of excess O₂, while heat rate is U-shaped in O₂ — dry-gas loss punishes running rich in air, CO and unburnt carbon punish starving it. Set the three weights (heat rate, NO_x, unburnt carbon) and watch the recommended operating point slide along the frontier. Run the two mandatory corners: all-weight-on-heat-rate (watch NO_x climb), all-weight-on-NO_x (watch heat rate pay). Then find a blend you would defend in a morning meeting — because the weights are **policy**, and “the optimiser says” is not a complete sentence.

In deployment this arrives as an *advisory*: setpoint nudges with reasons, expected recovery in kcal and rupees, the constraints respected, and the operator's right to decline with a reason (declines feed retraining). Closed-loop control only ever follows years of trusted advisory operation.

5.6 The missing instrument, and the one real deployment

Soft sensors (SIM-4, homework). The laboratory GCV arrives once a shift, hours late; every optimiser upstream spends the shift assuming the morning's coal. A soft sensor infers GCV every minute from mill behaviour — feeder rate against heat output, mill power, air temperatures — calibrated against the lab and running between its samples. Our June as-fired GCV averaged 3,061 kcal/kg; the swing *within* a day is what the optimisers never see without this.

Adani Udupi — read properly. The nearest real deployment to our shape: AVEVA PI historian with ML on top, boiler and turbine efficiency computed on 15-minute windows, health indices on critical pumps and motors, on Indian coal units. What it proves: the

historian-plus-models architecture works in production, here, on plants like ours. What it does not provide: audited rupee savings — so it cannot size our prize. Only our own bill can, and it just did. Proof of architecture, not proof of value; both halves said aloud.

5.7 The card

At the close of this block, every participant writes **one candidate home for the 176 kcal/kWh** on a card — specific, physical, from your own unit knowledge (“U9 condenser air ingress”, “HPH-6 bypassed since March”). The cards are collected and read against the pilot criteria in Chapter 7. The bill gave the size; your cards give the addresses.

Chapter 6 — Computer vision, and SafetyNet

6.1 The modest, true proposition

Computer vision does nothing a good engineer cannot do. It does what a good engineer cannot do **a hundred times a day, every day, without getting bored**. That is the entire proposition — volume, not brilliance — and it survives contact with reality, which grander claims do not.

Technically, nothing is new: a vision model is Chapter 2’s network with pixels for inputs and millions of dials, plus one idea — **convolution**, a small filter slid across the image reusing the same few weights, so the network learns “edge” once and finds it everywhere. Forward pass, error, backprop, training windows, contamination: everything you already know applies unchanged. And the historian lesson repeats with a lens on it: the dataset is the project. Thousands of labelled images from *your* site, consistent lighting, cameras maintained in coal dust — a dusty lens is a frozen sensor, and camera cleaning is now an instrument-maintenance route.

6.2 SafetyNet — a model trained at this station

SafetyNet is a per-worker PPE compliance system trained by an Assistant Engineer on a free Colab GPU — no procurement, no vendor. In the session it is demonstrated live from the presenter’s laptop through its Gradio interface (`python service/gradio_app.py` from the repository at github.com/pariharshyamu/Safetynet; the 19 MB model downloads itself on first run and everything after that is local — the demo needs no internet in the room):

Fact	Value
Architecture	YOLO11s, 120 epochs, 640 px
Training	free Colab T4, ~90 minutes, 2,801 annotated images (Roboflow construction-site-safety, CC BY 4.0)
Classes	10 — Hardhat, NO-Hardhat, Mask, NO-Mask, Safety Vest, NO-Safety Vest, Person, cone, machinery, vehicle

Fact	Value
Weights	19 MB, sha256-pinned
Detector metrics	mAP50 0.852 , mAP50-95 0.620, precision 0.921 , recall 0.787

Read the metrics honestly. Precision 0.921: when it flags a violation, it is right about nine times in ten. Recall 0.787: **it misses roughly one violation in five** — it is a screening aid, not a guarantee, and anyone deploying it must know that number.

6.3 The question nobody’s dataset answers

The dataset labels PPE as independent boxes, so a detector alone answers “is there a hardhat in this image?” — a question nobody asked. The site’s question is “**is that worker wearing one?**”, and bridging the two is an attribution problem the dataset does not label and mAP cannot measure. It is where SafetyNet’s real engineering lives: each PPE box is matched to at most one worker by three priors combined — containment, anatomical position, and **scale**, because a worker thirty metres away stands “inside” the bounding box of a worker three metres away, and containment alone happily awards the near worker the far worker’s hardhat. This is feature engineering — Chapter 4, stage 5 — in pixels.

Two design choices make it deployable. **It says “I don’t know”**. A worker under 64 pixels tall, clipped by the frame edge, or carrying contradictory detections (Hardhat at 0.62 and NO-Hardhat at 0.58 on the same head, inside a 0.10 margin) produces UNKNOWN, never an alarm. Absence of evidence is not evidence of a violation; systems that skip abstention get switched off in week two. **Policy lives outside the model**, in a plain configuration file: which items are required in which area, the minimum worker size, the ambiguity margin, how long a violation must persist before it becomes an event. Change site policy — masks required in enclosed spaces — and no retraining occurs. This is exactly the Model Builder’s alert-rule block: the detector reports what it sees; the policy decides what it means; the persistence rule debounces it. One principle, both worlds.

6.4 The industrial-relations question

The make-or-break issue is not technical. Unions will reasonably ask whether this is a safety system or a disciplinary system, and the answer is settled by what the first alert produces: **if it is a show-cause notice against a named individual, the programme is finished, permanently, on day one**. The design answer: zone-level statistics only, no identification, no name-and-shame, the union briefed *before* the first camera — with the miss-rate on the table, because a false alert against a named worker is a formal grievance with the programme’s name on it. If HR ever asks for the footage, the safety framing is dead, and so is the data source.

Never use PPE vision for: identifying individuals, disciplinary evidence, performance appraisal, or anything beyond zone statistics.

6.5 Where plant vision pays first

Ranked honestly, the proven rows are *thermal and volumetric* — physics measurements a camera happens to make: coal stockpile hot spots (thermal), drone stockpile volumetrics (the

recommended first project — outdoor, uncontroversial, safety-positive, and it produces a number finance already understands), switchyard and joint thermal rounds. Emerging: conveyor hot idlers and belt tracking, PPE zone statistics (governance first), flame vision. And the discipline of the whole course transfers: a vision alert is a **claim** — like day 255 — and claims want corroboration. A hot-idler camera plus a power-versus-tonnage residual on the same conveyor are two independent witnesses telling one story; that is when money moves.

Chapter 7 — From data to decisions

7.1 The other half of plant knowledge, and RAG

The historian holds the numbers; the documents hold everything else — manuals, SOPs, log books, trip reports, OEM letters — most of it findable only through one retiring engineer’s memory. A language model alone is next-word prediction trained on public text: it contains none of our documents and will answer questions about them confidently anyway. The danger mode is not gibberish; it is **fluent wrongness**, and near a permit-to-work, fluency without provenance is a hazard.

The fix is grounding, not size. **RAG** (retrieval-augmented generation): chunk our documents and index them by meaning, tagged with document, revision and page; retrieve the passages relevant to the question; answer *only* from what was retrieved, with the citation attached to every claim; and say “not found” when nothing relevant is retrieved — a feature, not a failure. The practical lever, measured on this course’s own retrieval bench (31 hand-marked questions): metadata. Clean section titles and revision tags moved recall@5 from 84% to **90%** — document hygiene, an AE-controllable input, beat the choice of model.

Mini-lab — Pocket RAG (5 minutes + homework). Open Pocket RAG from the landing page, load any PDF on your phone (or the built-in sample SOP), ask one question you know the answer to, and **tap the citation** to verify the quoted passage. Then ask a question the document does not answer, and watch it say so. Everything — parsing, indexing, answering — runs on the handset; the document never leaves it. In a government utility, that sentence is what makes this class of tool discussable at all. The trust checklist: citation present · document current · numbers recomputed.

7.2 Why these programmes fail

Every failure mode below was demonstrated on your own phone today. None is an algorithm failure; all are decisions nobody made.

What goes wrong	What prevents it
Training window touched the fault	Window ends before the judged period — procedurally
Winter window, summer alarms	Span the seasons; retrain after overhaul
Alert spam killed adoption	Fix the alert budget first, then spend it

What goes wrong	What prevents it
Symptom tags fed as inputs	Duty in, symptoms out
No named triage owner	Every advisory has a recipient before go-live
Model never retrained	Overhauls change “normal”; retraining is maintenance
Vendor black box	Demand the sensitivity table and the sister-machine run
Ungrounded AI answers	Citation or it does not count

7.3 The pilots

Criteria for a first pilot: tags already historised with a year of clean history; a failure mode this station has actually felt; money visible monthly from the bill; one named owner for triage; provable inside 90 days; no capex; a sister machine available as the negative control.

Pilot 1 — condition monitoring on the ID/FD/PA fans and BFPs of one 660 MW unit. Residual models on existing tags; alert budget two per month; success is defined in advance as *one advisory that becomes one work order before one failure*. On the simulator’s fan case this design yields 55–80 days of warning — quoted as simulation, not history. It proves the discipline: triage, budgets, owners.

Pilot 2 — the 176-kcal hunt. Continuous, tag-level loss attribution (SIM-2’s logic, automated hourly) on condenser vacuum, HP heaters and sprays for one unit. The bigger money — ₹22.87 crore a month of gross gap to localise — on a harder data problem that needs performance-grade quality. It goes second, and it shares Pilot 1’s historian export, quality gates and owner structure: phases, not rivals.

The 90-day shape. Days 1–30: tag list, one-year export, quality profile, training windows chosen and written down. Days 31–60: models trained, alert budget set, sister machines wired as negative controls. Days 61–90: live advisories into the morning meeting, every one triaged, closed and costed. What it needs from the organisation: a tag list, one engineer-week of export effort, a named triage owner, a monthly review — **no capex line**. The absence of a purchase order is the point; platform procurement is a phase-2 question, after triage discipline is proven.

7.4 The safety boundary

Policy, not preference, and written into the pilot charter on day one:

- **Never:** protection systems, interlocks, trip logic, PTW authority, closed-loop control of anything, identifying individuals on camera.
- **Only with controls:** operator-facing advisories with named owners; setpoint suggestions a human accepts or declines with a reason; zone-level safety statistics.
- **Freely:** historian analytics, residual monitoring, loss accounting, document retrieval with citations, training and simulation — everything in this course.

Cyber posture follows the same conservatism: everything in this course ran client-side; plant systems export data out and nothing connects in; historian exports follow the station’s existing

approval route; RAG stays on-device or on-premises; model files are versioned, hashed artefacts; and every advisory is logged with its evidence and outcome, because the audit trail is the programme's defence in any review.

7.5 Monday morning

Three actions, each with a named recipient:

1. Send the presenter your unit's five worst actors — machine, failure history, and which tags they carry.
2. Run the Model Builder on **one real historian export** from your own area. The CSV reader handles our format — semicolons, decimal commas, status strings and all.
3. Name the triage owner for your area: who receives an advisory if one arrives tomorrow morning?

The data is already in our historian. The decision layer is us.

Annex A — Pilot charter template (one page)

Pilot name: **Owner:**

Machines and tags: (list; attach the tag audit — fitted, missing, quality verdict per tag)

Failure modes named: (one line each; the chain from mechanism to trip)

Training window: from to (must end before the judged period; spans all seasons: yes / no) **Retrain triggers:** overhaul, sensor replacement, operating-regime change.

Alert budget: episodes/month. σ multiple: Persistence: hours. **Sister machine (negative control):**

Success criterion (decided now, not after): one advisory → one work order → one avoided failure, within 90 days. **Review:** monthly, with the bill on the table.

Boundary: advisories only; no DCS writes; no PTW authority; no individual identification.
Signed:

Annex B — The numbers, with provenance

Koradi 8–10, June 2026 (energy bill; Part-I FSA + F10 ECR; MERC MOD July 2026): capacity 1,980 MW (3×660); availability 72.5%; PLF 66.62%; gross generation 964.8 MU; net HR 2,442.18 vs 2,230 norm (+212.18); gross HR 2,272.45 vs 2,096.2 (+176.25 = ₹22.87 cr); aux 6.95% vs 6.0% (9.17 MU = ₹3.01 cr); net HR gap value ₹25.61 cr; as-fired GCV 3,061 kcal/kg; SFO 0.41 vs 0.50 ml/kWh; transit 0.49% vs 0.80%; variable cost ₹3.378 vs MOD ₹3.284/kWh; AFC ₹28.04 cr.

The synthetic fan case (PdM simulator / Model Builder, labelled synthetic throughout): 8,760 hourly rows; ambient Jan 21 – May 33.5 – monsoon 25.5 °C; CW lags ambient ~9 days; knee = $1.35 \cdot (CW - 27.5)^{1.45}$; fault onset day 200, ttf 110 days, shape 1.7; default model σ 2.05 °C (noise floor 0.89), band ± 8.2 °C, advisory day 255 (31% severity); +oilT – σ 0.60, day 230; window 0–300 – day 315; window 0–330 – never; winter window drift 4× its own band; motor-current-from-symptoms residual –22 A.

SafetyNet (github.com/pariharshyamu/Safetynet): YOLO11s, 120 epochs, Colab T4 $\approx$ 90 min, 2,801 images, 10 classes, 19 MB weights; mAP50 0.852, mAP50-95 0.620, precision 0.921, recall 0.787; abstention below 64 px person height and inside a 0.10 ambiguity margin; policy in compliance.yaml; temporal debouncing before any event.

Retrieval bench (this course's artefacts): 31 hand-marked questions; recall@5 84% dense, 84% keyword, 90% hybrid.

Annex C — Homework, with answer keys

All at pariharshyamu.github.io/simulator. **Theatre** Modules 1, 2 and 6 (k-NN, PCA, k-means — the same reading, three more machines). **PdM simulator**: run the HT motor, mill, BFP and transformer cases through all eight stages; each contains one deliberate trap — find it.

Simulation Lab: SIM-4 (GCV soft sensor) and SIM-5 (RUL as a band) complete Chapter 5.

Model Builder: repeat Experiments 1–4 on a second machine case, then on one of your own exports. **Pocket RAG**: one real SOP from your area, five questions, citations verified. Answers, where applicable, are in each artefact's own verdict panels — the point of the synthetic year is that the homework marks itself.